

ĐẠI HỌC AN GIANG

BỘ MÔN TIN HỌC

TÀI LIỆU GIẢNG DẠY

LẬP TRÌNH HỢP NGỮ

12/2014

A. LẬP TRÌNH HỢP NGỮ MIPS

I. Giới thiệu:

Hợp ngữ (Assembly) là ngôn ngữ lập trình bậc thấp, nó gồm tập các từ khóa và từ gọi nhớ rất gần với ngôn ngữ máy (machine code). Mỗi kiến trúc vi xử lý đều có tập lệnh (instruction set) riêng, do đó sẽ có hợp ngữ riêng dành cho kiến trúc đó. Ở đây, ta tập trung nghiên cứu về hợp ngữ dành cho kiến trúc MIPS.

1. Cấu trúc một chương trình hợp ngữ MIPS:

```
.data # khai báo biến sau chỉ thị này
...
.text # viết các lệnh sau chỉ thị này
main: # điểm bắt đầu của chương trình
...
```

2. Cách khai báo biến

tên_biến: kiểu_lưu_trữ_giá_trị

Các kiểu lưu trữ hỗ trợ: .word, .byte, .ascii, .asciiz, .space

Lưu ý: tên_biến (nhãn) phải theo sau bởi dấu hai chấm (:)

Ví dụ:

```
var1: .word 3 # số nguyên 4-byte có giá trị khởi tạo là 3
```

```
var2: .byte 'a','b' # mảng 2 phần tử, khởi tạo là a và b
```

```
var3: .space 40 # cấp 40-byte bộ nhớ, chưa được khởi tạo
```

```
char_array: .byte 'A':10 # mảng 10 ký tự được khởi tạo là 'A', có thể thay 'A' bằng 65
```

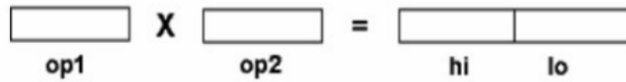
```
int_array: .word 0:30 # mảng 30 số nguyên được khởi tạo là 0
```

3. Các thanh ghi trong MIPS

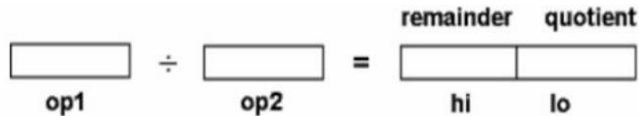
- Các thanh ghi đa năng

Số	Tên	Ý nghĩa
\$0	\$zero	Hằng số 0
\$1	\$at	Assembler Temporary
\$2-\$3	\$v0-\$v1	Giá trị trả về của hàm hoặc biểu thức
\$4-\$7	\$a0-\$a3	Các tham số của hàm
\$8-\$15	\$t0-\$t7	Thanh ghi tạm (không giữ giá trị trong quá trình gọi hàm)
\$16-\$23	\$s0-\$s7	Thanh ghi lưu trữ (giữ giá trị trong suốt quá trình gọi hàm)
\$24-\$25	\$t8-\$t9	Thanh ghi tạm
\$26-27	\$k0-\$k1	Dự trữ cho nhân HĐH
\$28	\$gp	Con trỏ toàn cục (global pointer)
\$29	\$sp	Con trỏ stack
\$30	\$fp	Con trỏ frame
\$31	\$ra	Địa chỉ trả về

- Thanh ghi HI và LO
 - Thao tác nhân của MIPS có kết quả chứa trong 2 thanh ghi HI và LO. Bit 0-31 thuộc LO và 32-63 thuộc HI.



- Thao tác chia của MIPS có thương số chứa trong các bit 0-31 thuộc LO và số dư chứa trong các bit 32-63 thuộc HI.



- Thanh ghi dấu phẩy động

MIPS sử dụng 32 thanh ghi dấu phẩy động để biểu diễn độ chính xác đơn của số thực. Các thanh ghi này có tên là : \$f0 – \$f31.
Để biểu diễn độ chính xác kép (double precision) thì MIPS sử dụng sự ghép đôi của 2 thanh ghi có độ chính xác đơn.

II. Tập lệnh hợp ngữ MIPS

1. Các lệnh luận lý (logic):

- Định dạng của lệnh or và ori (or immediate):

- Lệnh or:

or d, s, t # d là thanh ghi đích, s và t là các thanh ghi nguồn

Lệnh or thực hiện logic or trên từng bit giữa hai thanh ghi nguồn s và t, kết quả lưu vào thanh ghi đích d.

Lệnh or được dùng để copy nội dung của thanh ghi nguồn vào thanh ghi đích bằng cách thực hiện phép or với thanh ghi \$0

or d, s, \$0 # chép nội dung từ s vào d

- Lệnh ori :

ori d, s, const

d: thanh ghi đích

s: thanh ghi nguồn

const: là một số 16 bit (thể hiện ở dạng số thập phân hay số hex). Số này sẽ được bổ sung thêm các số 0 bên trái để đủ 32 bit.

OR Operation on Bits				
first operand	0	0	1	1
second operand	0	1	0	1
result	0	1	1	1

Lệnh ori sẽ thực hiện phép logic or theo từng bit giữa toán hạng nguồn s và const, kết quả được lưu vào thanh ghi đích d.

Ví dụ hai lệnh sau đây là tương đương nhau :

ori \$5,\$4,0x10 # const là số hex

ori \$5,\$4,16 # const là số thập phân

- Định dạng của lệnh and và andi (and immediate) :

- Lệnh and:

and d, s, t # d là thanh ghi đích, s và t là các thanh ghi nguồn

Lệnh and thực hiện logic and trên từng bit giữa hai thanh ghi nguồn s và t, kết quả lưu vào thanh ghi đích d.

- Lệnh andi:

andi d, s, const

d: thanh ghi đích

s: thanh ghi nguồn

const: là một số 16 bit (thể hiện ở dạng số thập phân hay số hex). Số này sẽ được bổ sung thêm các số 0 bên trái để đủ 32 bit.

AND Operation on Bits				
first operand	0	0	1	1
second operand	0	1	0	1
result	0	0	0	1

Lệnh andi sẽ thực hiện phép logic and theo từng bit giữa toán hạng nguồn s và const, kết quả được lưu vào thanh ghi đích d.

Ví dụ:

andi \$8,\$0,0x0002

hoặc andi \$8,\$0,0x2 # có thể bỏ các số 0 phía trước

- Định dạng của lệnh xor và xori (xor immediate) :

- Lệnh xor:

xor d, s, t # d là thanh ghi đích, s và t là các thanh ghi nguồn

Lệnh xor thực hiện logic xor trên từng bit giữa hai thanh ghi nguồn s và t, kết quả lưu vào thanh ghi đích d.

- Lệnh xori:

xori d, s, const

d: thanh ghi đích

s: thanh ghi nguồn

const: là một số 16 bit (thể hiện ở dạng số thập phân hay số hex). Số này sẽ được bổ sung thêm các số 0 bên trái để đủ 32 bit.

XOR operation on Bits				
first operand	0	0	1	1
second operand	0	1	0	1
result	0	1	1	0

Lệnh xori sẽ thực hiện phép logic xor theo từng bit giữa toán hạng nguồn s và const, kết quả được lưu vào thanh ghi đích d.

Ví dụ một chương trình đơn giản sử dụng các lệnh này:

```
.text
.globl main
main:
    ori    $15, $0, 0xFA5
    ori    $8, $15, 0x368F
    andi   $9, $15, 0x368F
    xori   $10, $15, 0x368F
```

- Định dạng lệnh nor:

nor d, s, t # d là thanh ghi đích, s và t là các thanh ghi nguồn

Lệnh nor thực hiện logic nor trên từng bit giữa hai thanh ghi nguồn s và t, kết quả lưu vào thanh ghi đích d. MIP không hỗ trợ lệnh nori (nor immediate)

NOR Operation on Bits				
first operand	0	0	1	1
second operand	0	1	0	1
result	1	0	0	0

Lệnh nor có thể dùng để đảo bit của một thanh ghi khi thực hiện với thanh ghi \$0:

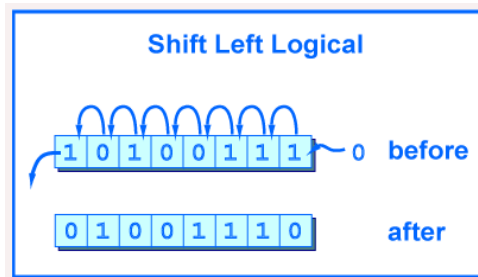
`nor d, s, $0` # đảo các bit của s để lưu vào d

2. Các lệnh dịch chuyển (shift instructions):

- Lệnh dịch trái luận lý sll (shift left logical):

`sll d, s, shift`

dịch chuyển sang trái *shift* bit của thanh ghi nguồn s và thêm các bit 0 vào bên phải. Kết quả được lưu vào thanh ghi đích d. Các thanh ghi d, s là thanh ghi độ dài 32 bit và $0 \leq \text{shift} < 32$.



Nếu thanh ghi nguồn biểu diễn số nguyên không dấu thì phép dịch sang trái n bit sẽ tương đương nhân toán hạng nguồn với 2^n .

Ví dụ:

nội dung được lưu vào 8 bit thấp của thanh ghi \$8, thực hiện dịch sang trái 2 bit.

`.text`

`.globl main`

`main:`

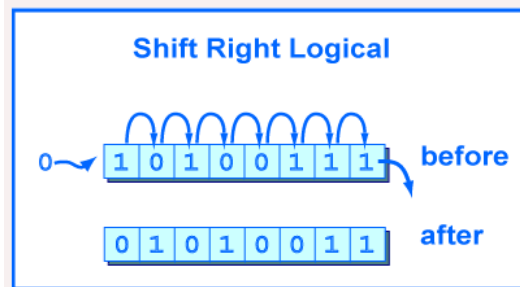
`ori $8, $0, 0x6F`

`sll $9, $8, 2`

- Lệnh dịch phải luận lý srl (shift right logical)

`srl d, s, shift`

dịch chuyển sang phải *shift* bit của thanh ghi nguồn s và thêm các bit 0 vào bên trái. Kết quả được lưu vào thanh ghi đích d. Các thanh ghi d, s là thanh ghi độ dài 32 bit và $0 \leq \text{shift} < 32$.



Nếu thanh ghi nguồn biểu diễn số nguyên không dấu thì phép dịch sang phải n bit sẽ tương đương chia toán hạng nguồn cho 2^n .

- Lệnh dịch phải số học sra (shift right arithmetic)
Lệnh dịch phải luận lý srl chỉ thực hiện đối với số không dấu. Khi thực hiện lệnh này đối với số có dấu (biểu diễn bằng bù 2) thì kết quả sẽ không đúng.

Ví dụ: số -16 biểu diễn dạng bù 2 bằng 16 bit như sau:

1111 1111 1111 0000

khi thực dịch phải logic 2 bit thì kết quả như sau:

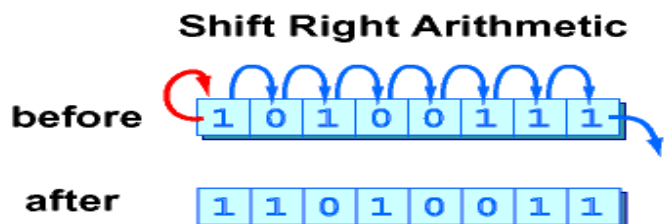
1111 1111 1111 0000 $\rightarrow$ 0011 1111 1111 1100

kết quả này không phải là biểu diễn của -4 (-16/4) mà là biểu diễn của một dương.

Do đó, đối với số có dấu cần thực hiện lệnh dịch phải số học có định dạng như sau:

sra d,s,shft # với $0 \leq \text{shft} < 31$

dịch sang phải shft vị trí của thanh ghi s và lưu kết quả vào thanh ghi d. Bít dấu được đưa trở lại bên trái của số cần dịch.



Tuy nhiên trong MIPS không có lệnh dịch trái số học bởi vì lệnh dịch trái luận lý đều đúng đối với số không dấu và số có dấu.

3. Các lệnh cộng (add) và trừ (sub):

- Lệnh add và addu:
add d, s, t
s,t: là các thanh ghi nguồn

d: là thanh ghi đích

cộng nội dung của s với t và lưu vào thanh ghi d với báo tràn (overflow) nếu có.

Tương tự lệnh:

addu d, s, t

cộng nội dung của s với t và lưu vào thanh ghi d với bỏ qua tràn (overflow) nếu có.

Ví dụ:

chương trình tính tổng +146 và (-82) được lưu dạng bù 2, kết quả lưu vào \$10

.text

.globl main

main:

ori \$7, \$0, 146 # đưa +146 vào \$7

ori \$8, \$0, 82 # đưa 82 vào \$8

nor \$8, \$8, \$0 # đảo bit (bù 1)

ori \$9, \$0, 1 # đưa 1 vào \$9

addu \$8, \$8, \$9 # \$8 = -82

addu \$10, \$7, \$8 # (+146) + (-82)

- Lệnh addi (add immediate) và addiu (add immediate unsigned):

addi d, s, const # báo tràn (overflow) nếu có

addiu d, s, const # bỏ qua tràn nếu có

const là số 16 bit, có thể được mở rộng lên 32 bit bằng cách chép bit dấu (bit 15) sang 16 bit có trọng số cao hơn (high-order bit) còn lại nếu số này biểu diễn bằng bù 2.

- Lệnh sub và subu:

sub d, s, t # báo tràn nếu có

subu d, s, t # bỏ qua tràn

các lệnh này thực hiện phép trừ $s - t$ và lưu kết quả vào thanh ghi d.

Tuy nhiên, trong MIPS không có lệnh subi và subiu bởi vì có thể thay thế bằng lệnh cộng tương ứng.

Ví dụ: để trừ 210 từ thanh ghi \$10 được thực hiện như sau

addiu \$8, \$10, -210

4. Các lệnh nhân và chia:

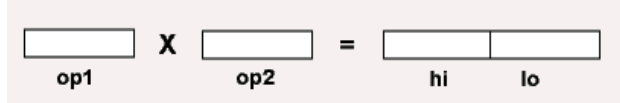
- Lệnh mult và multu:

mult s, t # nhân hai số có dấu (biểu diễn bằng bù 2)

multu s, t # nhân hai số không dấu

thực hiện phép nhân hai số chứa trong thanh ghi s và t. Kết quả lưu vào hai thanh ghi hi và lo (2 thanh ghi độ dài 32 bit) với lo lưu các bit thấp (bit 0 đến bit 31) và hi lưu các bit cao (bit 32 đến bit 63) của kết quả phép nhân.

Lưu ý: phép nhân không xảy ra hiện tượng tràn.



- Lệnh mfhi và mflo:

Do các thanh ghi hi và lo không thể sử dụng trong các phép toán số học và luận lý nên cần chép kết quả của phép nhân sang thanh ghi đa dụng.

mfhi d # chép nội dung thanh ghi hi sang thanh ghi d

mfli d # chép nội dung thanh ghi lo sang thanh ghi d

Ví dụ: chương trình tính biểu thức $5 * x - 74$

\$8 chứa x

\$9 chứa kết quả

.text

.globl main

main :

ori \$8, \$0, 12 # đưa x vào \$8 (giả sử x=12)

ori \$9, \$0, 5 # đưa 5 vào \$9

mult \$9, \$8 # lo = 5x

mflo \$9 # \$9 = 5x

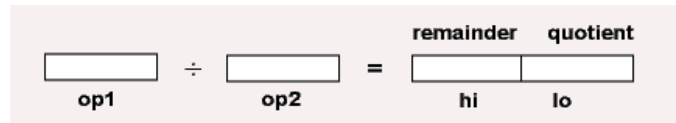
addiu \$9, \$9, -74 # \$9 = 5x - 74

- Lệnh div và divu:

div s, t # chia hai số có dấu (biểu diễn bằng bù 2)

divu s, t # chia hai số không dấu

thực hiện phép chia hai số chứa trong thanh ghi s và t. Kết quả thương số (s div t) lưu vào thanh ghi lo và số dư (s mod t) lưu vào thanh ghi hi.



Ví dụ: tính biểu thức $(y + x) / (y - x)$

chương trình tính $(y + x) / (y - x)$

Các thanh ghi sử dụng:

\$8 x

\$9 y

\$10 y+x

\$11 y-x

```
.text
.globl main
main:
    ori    $8, $0, 8      # đưa x vào $8 (giả sử x=8)
    ori    $9, $0, 36     # đưa y vào $9 (giả sử y=9)
    addu   $10, $9, $8     # $10 = (y+x)
    subu   $11, $9, $8     # $11 = (y-x)
    div    $10, $11        # hilo = (y+x)/(y-x)
    mflo   $10             # $10 = thương số
    mfhi   $11             # $11 = số dư
```

5. Các lệnh truy cập bộ nhớ

- Lệnh lw (load word)

lw d, off(b) # b là thanh ghi, off là số 16 bit (biểu diễn bù 2)
thực hiện chép một từ (word) tại địa chỉ b+off của bộ nhớ vào
thanh ghi \$d.

- Lệnh sw (store word)

sw t, off(b) # b là thanh ghi, off là số 16 bit (biểu diễn bù 2)
thực hiện chép một từ (word) vào địa chỉ b+off của bộ nhớ từ
thanh ghi \$t.

- Lệnh lui (load upper immediate)

lui d, const # d là thanh ghi, const là số 16 bit
lệnh này thực hiện chép nội dung 2 byte của const vào 2 byte cao
của thanh ghi \$d và xóa 2 byte thấp của \$d về 0.

Ví dụ: sao chép nội dung một từ tại địa chỉ 0x00040010 của bộ
nhớ vào thanh ghi \$12

```
lui $13, 0x0004
```

```
lw $12, 0x10($13)
```

- Ứng dụng: Mảng trong hợp ngữ MIPS

Ví dụ: khai báo một mảng có 6 phần tử số nguyên, mỗi phần tử được thể hiện là một từ (4 byte)

```
int data[] = {0, 1, 2, 3, 4, 5}; // ngôn ngữ C
```

```
int_data: .word 0, 1, 2, 3, 4, 5 # mảng trong MIPS
```

khi đó mảng này được tạo trong bộ nhớ như sau:

Memory	Addresses
00000005	00605014
00000004	00605010
00000003	0060500C
00000002	00605008
00000001	00605004
00000000	00605000

Ví dụ: chương trình tính $5x^2 - 12x + 97$

các thanh ghi sử dụng

\$10 chứa địa chỉ x

\$11 chứa x

\$12 chứa giá trị tạm thời

\$13 chứa giá trị tạm thời

đoạn dữ liệu .data luôn bắt đầu tại 0x10000000

```
.text
```

```
.globl main
```

```
main:
```

```
lui $10,0x1000 # khởi tạo địa chỉ x
```

```
lw $11,0($10) # gán x
```

```
ori $12,$0,97
```

```
ori $13,$0,12
```

```
mult $11,$13 # 12x
```

```
mflo $13
```

```
subu $12,$12,$13 # -12x +97
```

```
mult $11,$11 #  $x^2$ 
```

```
mflo $11
```

```
ori $13,$0,5    # 5
mult $11,$13    #  $5x^2$ 
mflo $13
addu $12,$12,$13 #  $5x^2-12x+97$ 
```

```
sw $12,4($10)
```

```
.data
x:    .word 17    # giả sử x=17
poly: .word 0     # vị trí lưu kết quả.
```

Ngoài ra, trong MIPS còn có các lệnh truy cập bộ nhớ (với byte và nửa từ - half word) nhưng ít sử dụng hơn lw và sw.

- Lệnh lb và lbu: thao tác với byte
 - lb t, off(b) # t và b là thanh ghi, off là số 16-bit
lệnh này chép 1 byte nội dung của bộ nhớ tại địa chỉ b+off vào byte thấp nhất (bit 0 – 7) của thanh ghi \$t và chép bit 7 sang các vị trí các bit còn lại (bit 8 – 31) của \$t. Lệnh lb được sử dụng khi muốn chuyển đổi một số có dấu 8 bit thành số có dấu 32 bit.
 - lbu t, off(b) # t và b là thanh ghi, off là số 16-bit
lệnh này chép 1 byte nội dung của bộ nhớ tại địa chỉ b+off vào byte thấp nhất (bit 0 – 7) của thanh ghi \$t và đưa 0 vào các vị trí còn lại (bit 8 – 31) của \$t. Lệnh lbu được sử dụng khi muốn chuyển đổi một số không dấu 8 bit thành số không dấu 32 bit.
- Lệnh sb

sb t, off(b) # t và b là thanh ghi, off là số 16-bit.
lệnh này lưu 8 bit thấp (bit 0 -7) của thanh ghi \$t vào bộ nhớ tại địa chỉ b+off. Các bit còn lại (bit 8-31) của thanh ghi \$t được bỏ qua. Sau khi thực hiện lệnh này thanh ghi \$t không thay đổi.
Trong MIPS không có lệnh sbu.
- Lệnh lh và lhu: thao tác nửa từ (halfword – 2 byte)
 - lh t, off(b) # t và b là thanh ghi, off là số 16-bit
lệnh này chép 2 byte nội dung của bộ nhớ bắt đầu tại địa chỉ b+off vào 2 byte thấp (bit 0 – 15) của thanh ghi \$t và chép bit 15 sang vị trí các bit còn lại (bit 16 – 31) của \$t.
 - lhu t, off(b) # t và b là thanh ghi, off là số 16-bit

lệnh này chép 2 byte nội dung của bộ nhớ bắt đầu tại địa chỉ $b+off$ vào 2 byte thấp (bit 0 – 15) của thanh ghi \$t và đưa 0 vào các vị trí còn lại (bit 16 – 31) của \$t.

- Lệnh sh

sh $t, off(b) \# t$ và b là thanh ghi, off là số 16-bit.

lệnh này lưu 2 byte thấp (bit 0 -15) của thanh ghi \$t vào bộ nhớ bắt đầu tại địa chỉ $b+off$. Các bit còn lại (bit 8-31) của thanh ghi \$t được bỏ qua. Sau khi thực hiện lệnh này thanh ghi \$t không thay đổi.

6. Các lệnh rẽ nhánh

- Lệnh j (jump)

j $target \#$ nhảy đến nhãn $target$

- Lệnh beq (branch on equal)

beq $u, v, addr \#$ nhảy đến $addr$ nếu $\$u = \v

- Lệnh bne (branch on not equal)

bne $u, v, addr \#$ nhảy đến $addr$ nếu $\$u \neq \v

- Các lệnh so sánh:

- blt (branch little than)

blt $u, v, addr \#$ nhảy đến $addr$ nếu $\$u < \v

- ble (branch little or equal)

ble $u, v, addr \#$ nhảy đến $addr$ nếu $\$u \leq \v

- bgt (branch greater than)

bgt $u, v, addr \#$ nhảy đến $addr$ nếu $\$u > \v

- bge (branch greater or equal)

bge $u, v, addr \#$ nhảy đến $addr$ nếu $\$u \geq \v

- Các lệnh nhảy (so sánh với 0)

bltz $s, label \#$ nhảy đến nhãn $label$ nếu thanh ghi $\$s < 0$

bgtz $s, label \#$ nhảy đến nhãn $label$ nếu thanh ghi $\$s > 0$

blez $s, label \#$ nhảy đến nhãn $label$ nếu thanh ghi $\$s \leq 0$

bgez $s, label \#$ nhảy đến nhãn $label$ nếu thanh ghi $\$s \geq 0$

Tuy nhiên, trong MIPS cũng không có lệnh beqz và bnez.

- Các lệnh thiết lập

slt $d, s, t \#$ if $(s < t) d = 1$ else $d = 0$

sltu $d, s, t \#$ dùng cho số không dấu

slti $t, s, const \#$ if $(s < const) t = 1$ else $t = 0$ với $const$ là số 16 bit.

sltiu $t, s, const \#$ dùng cho số không dấu

- Ứng dụng: minh họa một số cấu trúc điều kiện, vòng lặp

- Cấu trúc IF

Ví dụ 1:

if (a == b) c = d + e; else c = d - e;

Giả sử a, b, c, d, e được lưu lần lượt trong \$s0, ..., \$s4

Câu lệnh IF này tương ứng với các lệnh hợp ngữ sau:

```
bne $s0, $s1, else
addu $s2, $s3, $s4
j exit
else: subu $s2, $s3, $s4
exit: ...
```

Ví dụ 2:

if ((\$s1 > 0) && (\$s2 < 0)) {\$s3++;}

Các lệnh hợp ngữ tương ứng là:

```
blez $s1, next      # skip if false
bgez $s2, next      # skip if false
addiu $s3, $s3, 1    # both are true
next: ...
```

- Vòng lặp

Ví dụ:

i = 0; while (A[i] != k) i = i+1;

Giả sử địa chỉ của A được lưu trong \$s0 và i, k được lưu lần lượt trong \$s1, \$s2.

Memory

...	
A[i]	A+4×i
...	
A[2]	A+8
A[1]	A+4
A[0]	A
...	

Các lệnh hợp ngữ tương ứng:

```
xor $s1, $s1, $s1    # i = 0
move $t0, $s0        # $t0 = address A
```

```

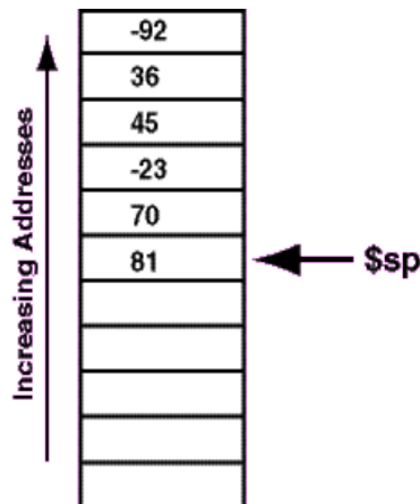
loop:    lw $t1, 0($t0)      # $t1 = A[i]
        beq $t1, $s2, exit  # exit if (A[i]== k)
        addiu $s1, $s1, 1   # i = i+1
        sll $t0, $s1, 2     # $t0 = 4*i
        addu $t0, $s0, $t0  # $t0 = address A[i]
        j loop
exit: ...

```

7. Cấu trúc Stack

Stack (ngăn xếp) là vùng nhớ đặc biệt được truy cập theo cơ chế “vào trước ra sau” (LIFO – Last In First Out), nghĩa là dữ liệu nào đưa vào sau sẽ được lấy ra trước.

Hình bên dưới là cấu trúc stack trong bộ nhớ, mỗi phần tử có kích thước một word (32-bit).



Thanh ghi \$sp đóng vai trò là con trỏ ngăn xếp (stack pointer), luôn chỉ đến đỉnh của stack. Stack phát triển theo chiều giảm của địa chỉ vùng nhớ (đỉnh của stack luôn có địa chỉ thấp).

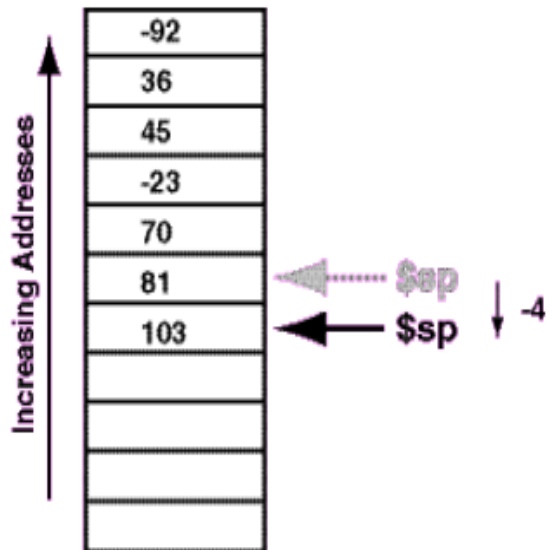
Hai thao tác cơ bản trong stack là push (đưa một phần tử vào stack) và pop (lấy một phần tử ra khỏi stack). Cơ chế như sau:

- Push: giảm \$sp đi 4, lưu giá trị vào ô nhớ mà \$sp chỉ đến.

Ví dụ: push vào stack giá trị trong \$t0

```
subu $sp, $sp, 4
```

```
sw $t0, ($sp)
```

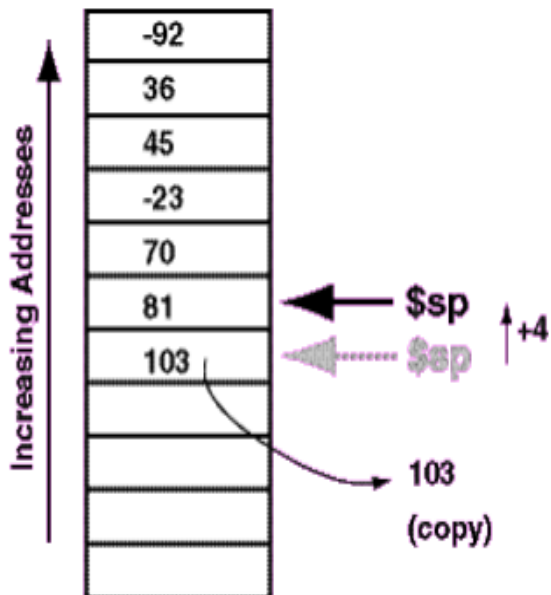


- Pop: copy giá trị trong vùng nhớ được chỉ đến bởi \$sp, cộng 4 vào \$sp.

Ví dụ: pop từ stack ra \$t0

lw \$t0, (\$sp)

addu \$sp, \$sp, 4



8. Thủ tục

MIPS hỗ trợ một số thanh ghi để lưu trữ các dữ liệu phục vụ cho thủ tục:

- Đôi số \$a0, \$a1, \$a2, \$a3
- Kết quả trả về \$v0, \$v1
- Biến cục bộ \$s0, \$s1, ... , \$s7

- Địa chỉ quay về \$ra

Cấu trúc của một thủ tục:

Đầu thủ tục

entry_label:

addi \$sp,\$sp, -framesize # khai báo kích thước cho stack

sw \$ra, framesize-4(\$sp) # cất địa chỉ trở về của thủ tục trong \$ra vào ngăn xếp (dùng khi gọi hàm lồng nhau)

Lưu tạm các thanh ghi khác (nếu cần)

Thân thủ tục ...

(có thể gọi các thủ tục khác...)

Cuối thủ tục

Phục hồi các thanh ghi khác (nếu cần)

lw \$ra, framesize-4(\$sp) # lấy địa chỉ trở về ra \$ra

addi \$sp,\$sp, framesize

jr \$ra

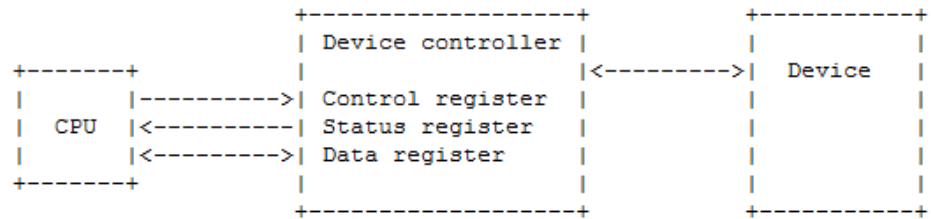
Gọi thủ tục: jal entry_label

B. GIAO TIẾP VỚI CÁC THIẾT BỊ

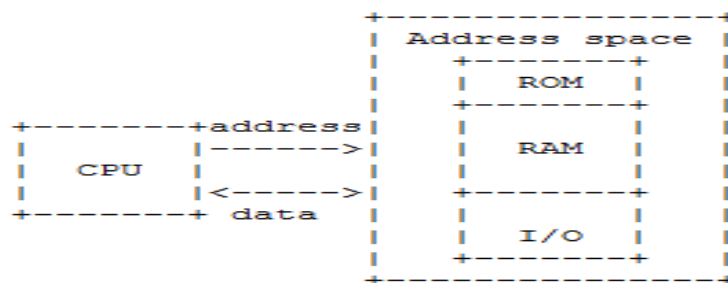
I. Giới thiệu:

Các thiết bị xuất nhập (I/O device) giao tiếp với CPU thông qua các thanh ghi xuất nhập (I/O register), có ba loại:

- Thanh ghi trạng thái (Status register): cung cấp thông tin trạng thái của thiết bị, thanh ghi này chỉ có thể đọc. CPU có thể đọc thanh ghi này nhưng không thể thay đổi.
- Thanh ghi điều khiển (Control register): CPU có thể ghi vào thanh ghi này để điều khiển thiết bị.
- Thanh ghi dữ liệu (Data register): có thể dùng để đọc dữ liệu hoặc ghi dữ liệu với thiết bị xuất nhập.



Đối với kiến trúc vi xử lý MIPS, CPU dùng không gian bộ nhớ để giao tiếp với thiết bị nhập xuất. Khi CPU đọc hoặc ghi vào các vùng nhớ này là đang đọc hoặc ghi vào các thanh ghi xuất nhập của các thiết bị xuất nhập. Kiểu kiến trúc này gọi là kiến trúc Nhập/Xuất ánh xạ Bộ Nhớ (*memory-mapped I/O*). Để truy cập đến thiết bị nhập / xuất có thể dùng các lệnh truy cập bộ nhớ, không có lệnh riêng dùng cho thiết bị xuất nhập.



II. Giao tiếp với trình điều khiển bàn phím:

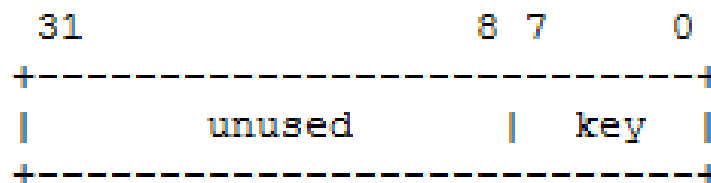
Trình điều khiển bàn phím chứa hai thanh ghi:

- Thanh ghi nhận dữ liệu (*receiver data register*) độ dài 32 bit nhưng chỉ có 8 bit thấp được sử dụng để chứa mã ASCII/ISO của phím được nhấn sau cùng. Thanh ghi này được định vị tại địa chỉ 0xffff0004. Thanh ghi này là thanh ghi chỉ đọc và có thể được truy cập bằng lệnh load.

mã ASCII của phím được nhấn cuối cùng được lưu vào byte thấp của \$t0

và xóa các bit còn lại của \$t0

lbu \$t0, 0xffff0004



Địa chỉ 0xffff0004 cũng được gán bằng hằng CONS_RECEIVER_DATA. Do đó có thể dùng tên hằng này thay thế khi truy cập thanh ghi này.

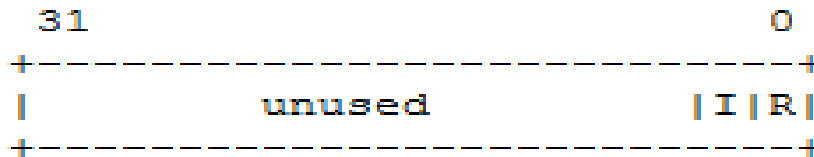
CONS_RECEIVER_DATA = 0xffff0004

mã ASCII của phím được nhấn cuối cùng được lưu vào byte thấp của \$t0

và xóa các bit còn lại của \$t0

lbu \$t0, CONS_RECEIVER_DATA

- Thanh ghi điều khiển (*receiver control register*) : có độ dài 32 bit được định vị tại địa chỉ 0xffff0000 nhưng chỉ có bit 0 và bit 1 được sử dụng



Bit 0 (R) là bit báo trạng thái sẵn sàng (*ready*) của bàn phím. Trình điều khiển bàn phím sẽ thiết lập 1 khi một phím được nhấn và được tự động xóa đi khi thanh ghi nhận dữ liệu được đọc.

Bit 1 (I) là bit báo ngắt quãng (*interrupt-enable*): khi có ngắt quãng bàn phím xảy ra, bit này sẽ được thiết lập giá trị 1 bằng phần mềm.

Để kiểm tra thiết bị có sẵn sàng hay không? CPU sẽ liên tục kiểm tra (polling) bit 0 của thanh ghi điều khiển. Chương trình polling thể hiện như sau:

kí tự nhập từ bàn phím sẽ được lưu vào \$v0

lui \$t0,0xffff #ffff0000

waitloop:

lw \$t1,0(\$t0) #control

andi \$t1,\$t1,0x0001

beq \$t1,\$0,waitloop

lw \$v0,4(\$t0) #data

III. Giao tiếp với trình điều khiển màn hình:

Trình điều khiển màn hình cũng hoạt động tương tự như trình điều khiển bàn phím, có hai thanh ghi:

- Thanh ghi điều khiển truyền (*transmitter control register*) được định vị tại địa chỉ 0xffff0008. Bit 0 của thanh ghi này báo trạng thái sẵn sàng (ready bit), giá trị 1 báo màn hình sẵn sàng nhận một kí tự khác.
- Thanh ghi dữ liệu truyền (*transmitter data register*) được định vị tại địa chỉ 0xffff000c. Kí tự được thể hiện ra màn hình được ghi vào thanh ghi này.

Khi màn hình thể hiện xong kí tự, trình điều khiển màn hình sẽ thiết lập bit trạng thái sẵn sàng (ready bit) của thanh ghi điều khiển truyền thành 1.

Tương tự như bàn phím, đối với màn hình CPU cũng sẽ liên tục kiểm tra (polling) bit 0 của thanh ghi điều khiển để xem thiết bị có sẵn sàng hay chưa?

Chương trình polling thể hiện như sau:

xuất kí tự trong thanh ghi \$a0 ra màn hình

```
lui    $t0,0xffff    #ffff0000
```

waitloop:

```
lw     $t1,8($t0)    #control
```

```
andi   $t1,$t1,0x0001
```

```
beq    $t1,$0,waitloop
```

```
sw     $a0,12($t0)   #data
```

IV. Hỗ trợ lệnh gọi hệ thống (System call):

Các chương trình giả lập (như PCSPIM) hỗ trợ lệnh syscall để chuyển quyền điều khiển cho HĐH. Sau đó, HĐH sẽ xem giá trị thanh ghi \$v0 để xác định xem chương trình muốn nó làm việc gì.

Dịch vụ	Giá trị trong \$v0	Đối số	Kết quả
print_int	1	\$a0 = integer	
print_float	2	\$f12 = float	
print_double	3	\$f12 = double	
print_string	4	\$a0 = string	
read_int	5		integer (trong \$v0)
read_float	6		float (trong \$f0)
read_double	7		double (trong \$f0)
read_string	8	\$a0 = buffer, \$a1 = length	
sbrk	9	\$a0 = amount	address (trong \$v0)
exit	10		
print_character	11	\$a0 = char	
read_character	12		char (trong \$v0)

Ví dụ 1: in chuỗi ra màn hình

```
.data # khai báo data segment
```

```
str:    .asciiz "hello world"
```

```
.text
```

```
.globl main
```

```
main:   # nhận main cho vi xử lý biết nơi thực thi lệnh đầu tiên
```

```
la $a0, str    # tải địa chỉ của nhãn str vào thanh ghi $a0
```

```

addi $v0, $zero, 4    # đưa giá trị 4 vào thanh ghi $v0
syscall
addi $v0, $zero, 10
syscall

```

Ví dụ 2: chương trình tính và in ra tổng của hai số nhập vào từ bàn phím

```

#      $t0: chứa số thứ nhất
#      $t1: chứa số thứ hai
#      $t2: chứa tổng của $t0 và $t1

```

```

# nhập số thứ nhất lưu vào $t0.
li $v0, 5
syscall
move $t0, $v0

```

```

# nhập số thứ hai lưu vào $t1.
li $v0, 5
syscall
move $t1, $v0

```

```

# tính tổng hai số lưu vào $t2
add $t2, $t0, $t1

```

```

# in tổng hai số
move $a0, $t2
li $v0, 1
syscall

```

```

li $v0, 10
syscall

```

BÀI TẬP

Hãy viết chương trình hợp ngữ MIPS để giải quyết các bài toán sau:

1. Nhập vào một chuỗi, xuất lại chuỗi đó ra màn hình

Ví dụ:

Nhap mot chuoi: Hello

Chuoi da nhap: Hello

2. Nhập vào một ký tự, xuất ra ký tự liền trước và liền sau.

Ví dụ:

Nhap mot ky tu: b

Ky tu lien truoc: a

Ky tu lien sau: c

3. Nhập vào một ký tự hoa, in ra ký tự thường.

Ví dụ:

Nhap mot ky tu: A

Ky tu thuong: a

4. Nhập từ bàn phím 2 số nguyên, tính tổng, hiệu, tích, thương của 2 số.

Ví dụ:

Nhap so thu nhât: 7

Nhap so thu hai: 4

Tong: 11

Hieu: 3

Tich: 28

Thuong: 1 du 3

5. Nhập vào 2 số nguyên, xuất ra số lớn hơn.

Ví dụ:

Nhap so thu nhât: 6

Nhap so thu hai: 9

So lon hon la: 9

6. Nhập một ký tự từ bàn phím. Nếu ký tự vừa nhập thuộc [0-9], [a-z], [A-Z] thì xuất ra

màn hình ký tự đó và loại của ký tự đó (số, chữ thường, chữ hoa).

Ví dụ:

Nhập vào một ký tự: 5

Ký tự vừa nhập: 5 là số

Nhập vào một ký tự : f

Ký tự vừa nhập : f là chữ thường

Nhập vào một ký tự : D

Ký tự vừa nhập : D là chữ hoa

7. Nhập một mảng các số nguyên n phần tử, xuất mảng đó ra màn hình.

Ví dụ:

Nhap n: 5

[0] = 4

[1] = 2

[2] = 7

[3] = 9

[4] = 3

Mang vua nhap: 4 2 7 9 3

8. Nhập vào một số nguyên n, tính tổng từ 1 đến n.

Ví dụ:

Nhap mot so: 4

Tong tu 1 den 4 la: 10

9. Nhập vào một chuỗi. Tính chiều dài của chuỗi.

Ví dụ:

Nhap mot chuoi: HCMUS

Chieu dai cua chuoi: 5

10. Nhập vào mảng 1 chiều n số nguyên. Xuất giá trị lớn nhất và nhỏ nhất.

Ví dụ:

Nhap n: 3

[0] = 4

[1] = 2

[2] = 7

Gia tri nho nhat: 2

Gia tri lon nhat: 7

11. Nhập vào một chuỗi, xuất ra chuỗi ngược.

Ví dụ:

Nhap vao mot chuoi: hello

Chuoi nguoc la: olleh

12. Viết lại bài 1 dưới dạng thủ tục.

Hướng dẫn: Hàm xuất chuỗi có dạng sau PRINT(&buf), &buf là địa chỉ của vùng nhớ chứa chuỗi.

13. Viết lại bài 4 dưới dạng thủ tục.

Hướng dẫn: Hàm tính tổng có dạng sau SUM(X, Y, Z), trong đó X, Y là 2 số nguyên, Z là tổng của 2 số.